习题课 3 笔记：
2022 年 9 月 30 日

提醒事项：
- 下周三（10 月 5 日）的 MQ4
- PS1 下周三截止

第 4 次课回顾：简单程序，二进制数简介

1. 简单程序
- 猜和试算法是通过对所有可能情况进行穷举来找到问题解法的一种方法。

- 猜测解法 -> 评估猜测 -> 对猜测进行有根据的调整并重复……
- 重复这些步骤，直到你找到解法或用尽了所有可能的解法。
- 示例程序：猜测平方根或立方根。

2. 二进制数简介
- 计算机使用二进制数。
- 所有信息都存储在两种状态中——要么是 0，要么是 1。
- 二进制数高效且易于进行操作。
- 一串二进制数，例如 00110011，称为一个比特序列。
- 十进制数可以转换为二进制数，反之亦然。

前八位是 2 的幂：128, 64, 32, 16, 8, 4, 2, 1。因此，在前八位中我们可以
存储的最大数是（不包括）256。

示例：将十进制数 56 转换为二进制表示。56 = 00111000

示例 2：将 00011001 转换为十进制。
00011001 = 1*1 + 1*8 + 1*16 = 25

第五讲回顾：浮点数、分数和近似算法
1. 浮点数
- Python 使用“浮点数”来近似表示实数。
- 对浮点数进行运算会产生非常小的误差。
- 许多小错误会变成大错误。

2. 分数与小数近似
- 我们使用同样的方法，通过将 2 提升到某个负数的幂来在二进制中存储分数。

最终，计算机用位来表示所有内容。因此，小数点后有很多位数的数字通常会被近似。

因此，在比较和处理浮点数时要小心。

3. 近似算法

- 类似于猜测和检查，但目标是要找到一个被认为是“足够好”的答案，而不一定是精确的。

- 猜测一个答案 -> 检查它是否“足够好” -> 如果不是，做一个有根据的更改你的猜测 -> 重复直到你的猜测是“足够好”
- 关键参数：增量、epsilon、猜测次数等…
- 记住要考虑如果超过接近足够的停止条件会发生什么——不希望出现无限循环。

第六次课回顾：二分查找、牛顿-拉夫森法

1. 二分查找

● 应用于具有内在顺序的可能答案范围的问题（例如一个有序的数字列表）的搜索算法。

● 简单二分搜索算法的步骤： ○ 猜测区间的中点 ○ 如果不是答案，检查答案是否大于或小于中点 ○ 改变区间 ○ 重复

● 这种方法在每个阶段将可能答案的集合减半 → 对数增长特性 → 更高效的算法

2. 牛顿-拉夫森

● 一元多项式根的通用近似算法
● 给定多项式函数 p(x)，目标是求解 r 使得 p(r) = 0。
● N-R 证明了：
  ○ 如果 g 是根 r 的近似值，那么 g - p(g)/p’(g) 是更好的近似值，其中 p’ 是 p 的导数。

第 7 讲：函数和作用域函数

- 函数将计算封装在黑盒中。
- 它们允许我们重用代码并以更简洁的方式编写程序。
- 函数接收输入并返回输出。
- 输入作为函数的参数，输出使用 return 语句返回。

- 调用函数 My_output = function_name(arg1, arg2, …, argN)
- 调用时，整个函数被替换为返回值
- print 与 return
  - print：对用户而言，仅显示一个值
  - return：对计算机而言，允许将值从函数发送回代码的其他部分

    - 在执行 return 语句后，函数中不会有任何其他操作被执行。
    - Python 的默认返回值是 None。

作用域

- 变量赋值在符号表或栈帧中跟踪，将变量名映射到它们的值

- 当函数被调用时，会创建一个新的栈帧。
- 当函数返回时，栈帧被弹出/销毁
- 我的 Python 导师对此做了很好的可视化 https://pythontutor.com/。

麻省理工学院开放课程资料
https://ocw.mit.edu

6.100L 使用 Python 的计算机科学和编程导论

2022 年秋季

有关引用这些材料或我们使用条款的信息，请访问：https://ocw.mit.edu/terms